Big Data with Hadoop

Assessment

Name: Muniba Javaid

Section 1: HDFS Using the Command Line

Commands:

- `Wget`
- `https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv`
- `ls`
- `hadoop fs -copyFromLocal penguins.csv /user/maria_dev/datasets/penguins.csv`

Output:

```
sandbox-hdp login: maria_dev
maria_dev@sandbox-hdp.hortonworks.com's password:
Last login: Thu Oct 15 09:34:32 2026 from 172.18.0.2
[maria_dev@sandbox-hdp ~]$ wget https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv
--2026-10-15 10:10:25--  https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 15241 (15K) [text/plain]
Saving to: 'penguins.csv'

100%[=============================================================================================================>] 15,241      --.-K/s   in 0.01s

2026-10-15 10:10:25 (1.16 MB/s) - 'penguins.csv' saved [15241/15241]

[maria_dev@sandbox-hdp ~]$ ls
1-spark.py  8836201  iris.csv  penguins.csv
[maria_dev@sandbox-hdp ~]$ hdfs fs -copyFromLocal penguins.csv /user/maria_dev/datasets/penguins.csv
Error: Could not find or load main class fs
[maria_dev@sandbox-hdp ~]$ hadoop fs -copyFromLocal penguins.csv /user/maria_dev/datasets/penguins.csv
[maria_dev@sandbox-hdp ~]$
```

Figure 1: Downloading the dataset and copying it to HDFS

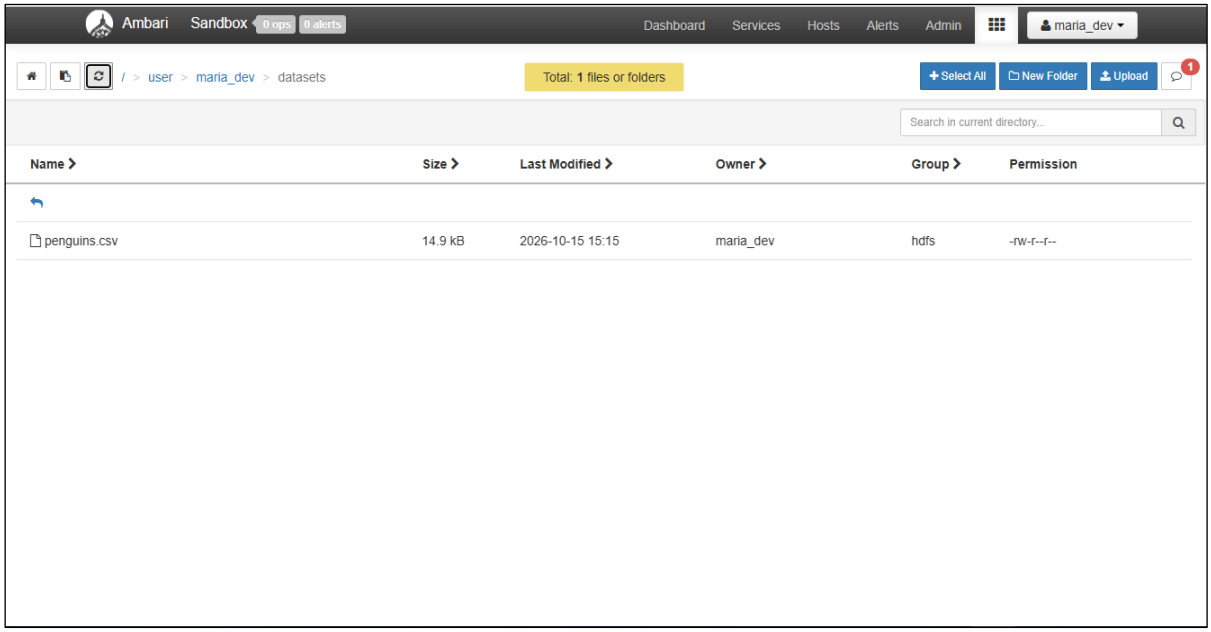


Figure 2: penguins.csv inside /user/maria_dev/datasets

Section 2: Hive Table and HiveQL

Creating the database and table:

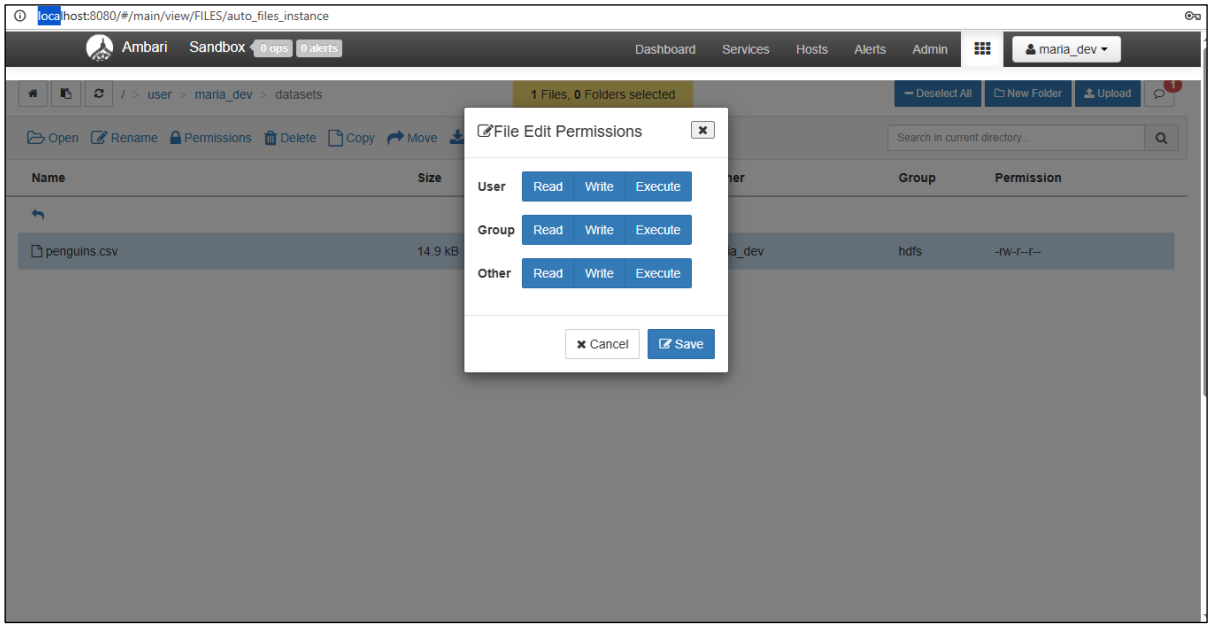


Figure 3: Changing the permissions of penguins.csv

```
CREATE DATABASE penguins_db;
```

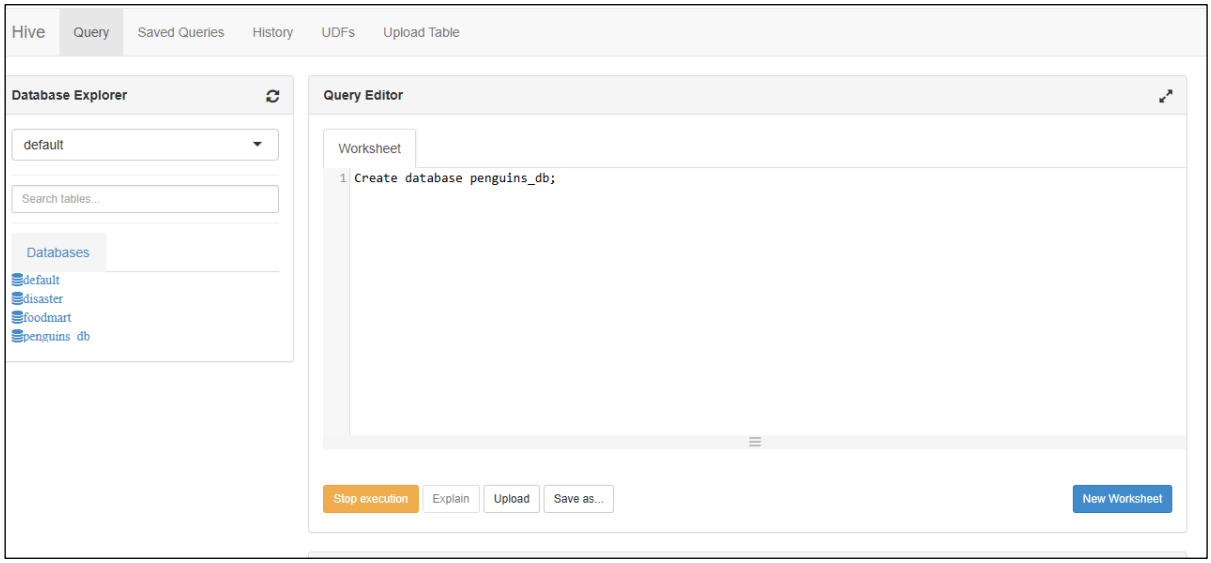


Figure 4: penguins_db created in Hive

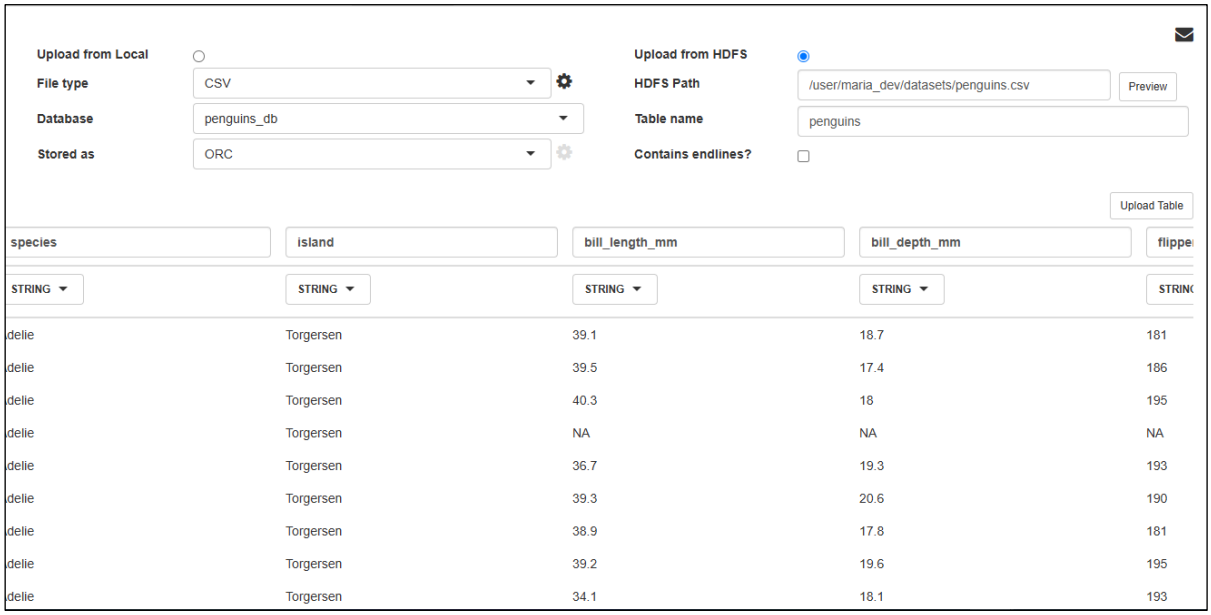


Figure 5: Uploading the CSV from HDFS as a table

3a: Previewing The Data

```
SELECT * FROM penguins LIMIT 10;
```

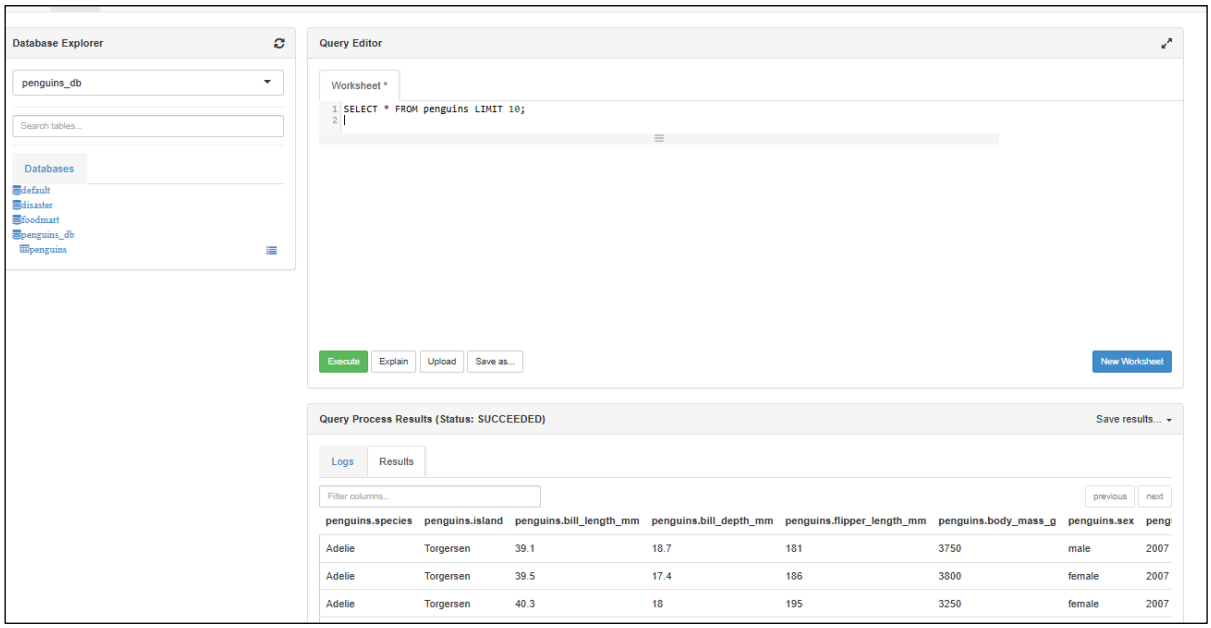


Figure 6: Output of 3a

3b: Records per species and island

```
SELECT species, island, COUNT(*) AS num_records
FROM penguins
GROUP BY species, island
ORDER BY species, island;
```

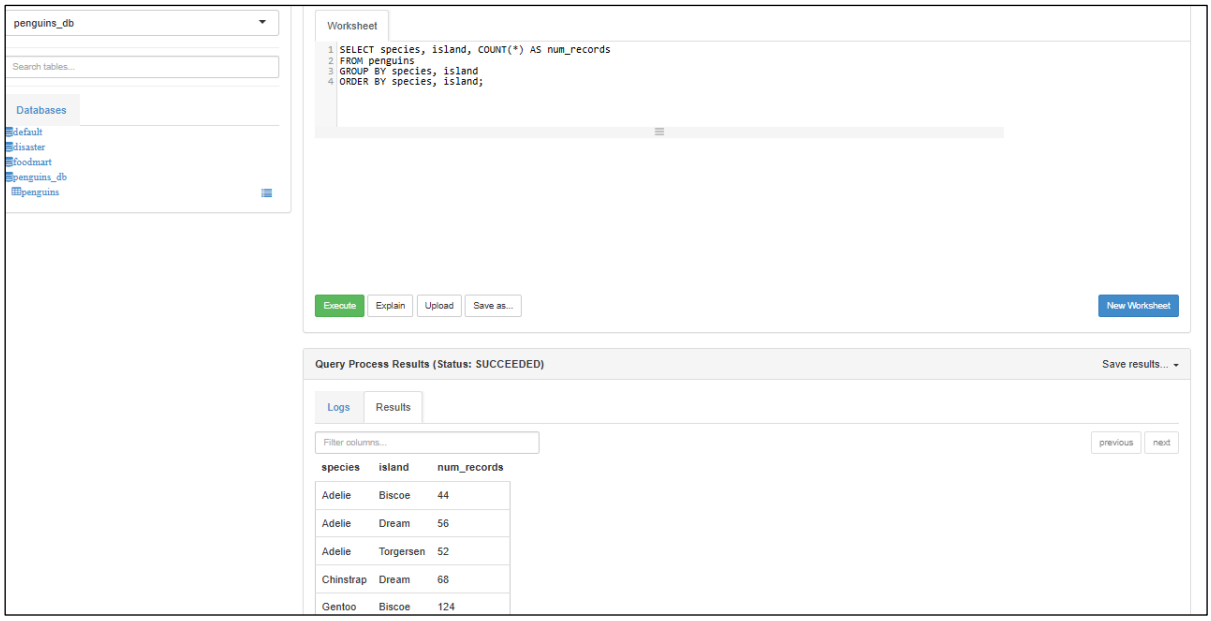


Figure 7: Output of 3b

3c: Average body mass and flipper length per species

```
SELECT species,
ROUND(AVG(body_mass_g), 2) AS avg_body_mass_g,
ROUND(AVG(flipper_length_mm), 2) AS avg_flipper_length_mm
FROM penguins
GROUP BY species;
```

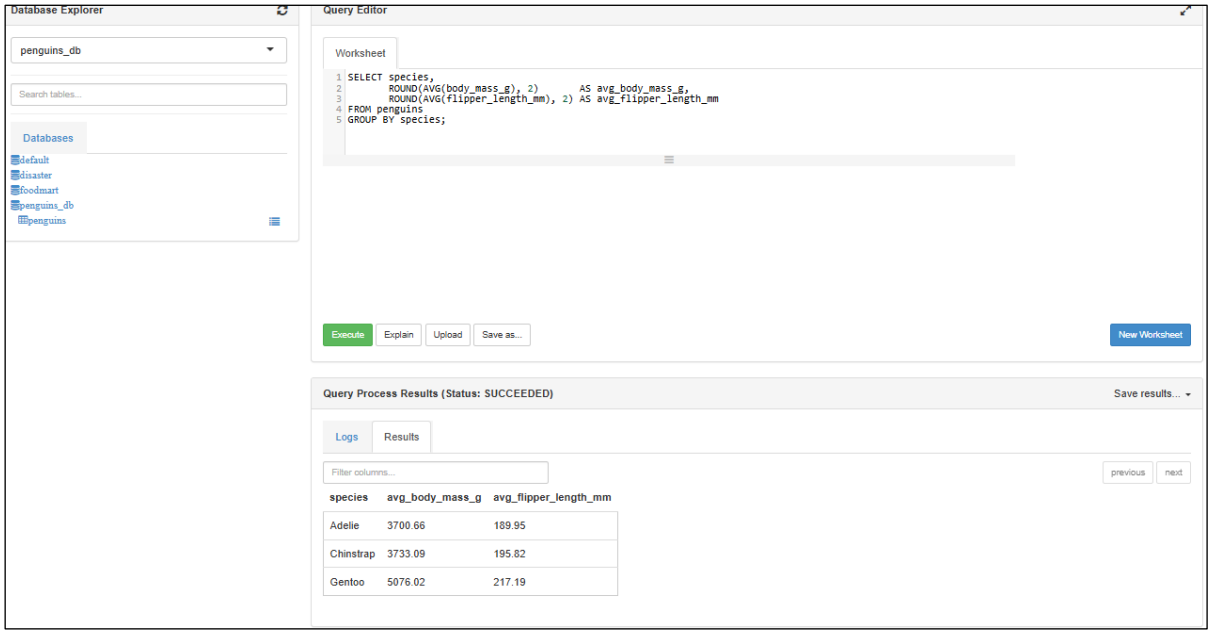


Figure 8: Output of 3c

3d: Missing values

```
SELECT
SUM(CASE WHEN bill_length_mm IS NULL THEN 1 ELSE 0 END) AS
missing_bill_length_mm,
SUM(CASE WHEN flipper_length_mm IS NULL THEN 1 ELSE 0 END) AS
missing_flipper_length_mm,
SUM(CASE WHEN body_mass_g IS NULL THEN 1 ELSE 0 END) AS
missing_body_mass_g,
SUM(CASE WHEN sex IS NULL OR sex = '' THEN 1 ELSE 0 END) AS missing_sex
FROM penguins;
```

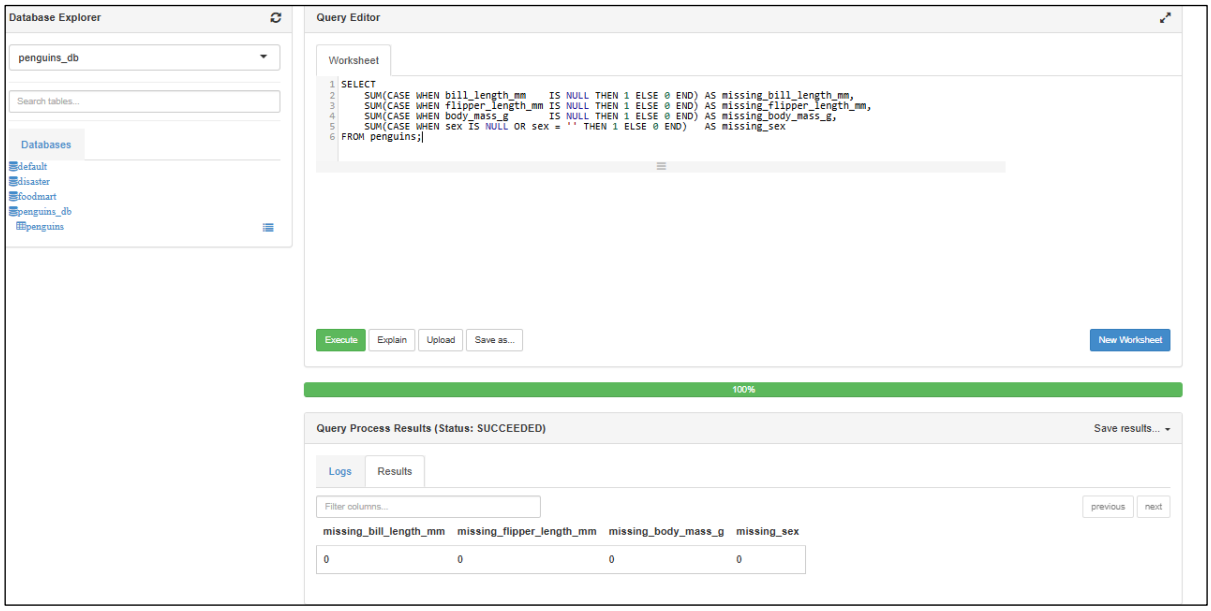


Figure 9: Output of 3d

Section 3: ELT with WebHDFS InsecureClient

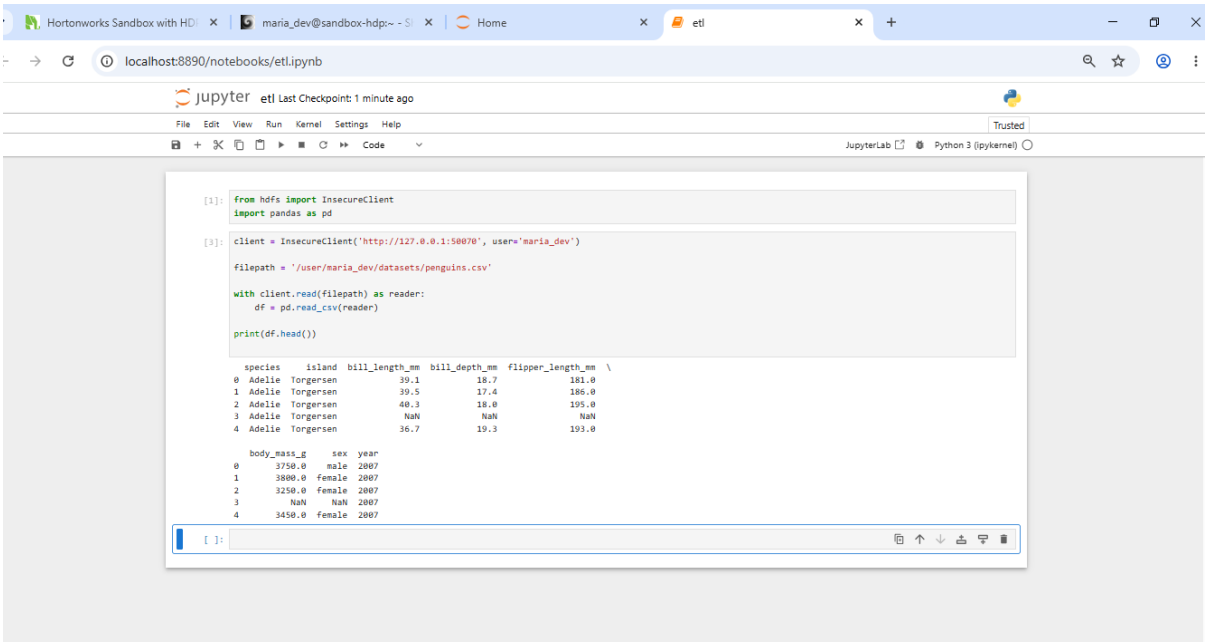


Figure 10: Jupyter Notebook connected to the sandbox

[4]:
```
from hdfs import InsecureClient
import pandas as pd
```

1. Load penguins.csv from HDFS into Pandas

[5]:
```
client = InsecureClient('http://127.0.0.1:50070', user='maria_dev')

filepath = '/user/maria_dev/datasets/penguins.csv'

with client.read(filepath) as reader:
    df = pd.read_csv(reader)

print(df.head())
```

```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex  year
0       3750.0    male  2007
1       3800.0  female  2007
2       3250.0  female  2007
3          NaN     NaN  2007
4       3450.0  female  2007
```

[6]:
```
print('Shape:', df.shape)
df.info()
```

```
Shape: (344, 8)
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
 7   year               344 non-null    int64
dtypes: float64(4), int64(1), str(3)
memory usage: 27.6 KB
```

Figure 11: Loading the data into Pandas

[7]:
```
df.describe(include='all')
```

[7]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
count	344	344	342.000000	342.000000	342.000000	342.000000	333	344.000000
unique	3	3	NaN	NaN	NaN	NaN	2	NaN
top	Adelie	Biscoe	NaN	NaN	NaN	NaN	male	NaN
freq	152	168	NaN	NaN	NaN	NaN	168	NaN
mean	NaN	NaN	43.921930	17.151170	200.915205	4201.754386	NaN	2008.029070
std	NaN	NaN	5.459584	1.974793	14.061714	801.954536	NaN	0.818356
min	NaN	NaN	32.100000	13.100000	172.000000	2700.000000	NaN	2007.000000
25%	NaN	NaN	39.225000	15.600000	190.000000	3550.000000	NaN	2007.000000
50%	NaN	NaN	44.450000	17.300000	197.000000	4050.000000	NaN	2008.000000
75%	NaN	NaN	48.500000	18.700000	213.000000	4750.000000	NaN	2009.000000
max	NaN	NaN	59.600000	21.500000	231.000000	6300.000000	NaN	2009.000000

[8]:
```
print('Missing values per column:')
df.isnull().sum()
```

```
Missing values per column:
```

[8]:
```
species               0
island                0
bill_length_mm        2
bill_depth_mm         2
flipper_length_mm     2
body_mass_g           2
sex                  11
year                  0
dtype: int64
```

Figure 12: describe() and missing values

2. Clean and preprocess

[9]:
```
clean = df.copy()

# Drop rows missing ALL body measurements (no useful data)
measure_cols = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
clean = clean.dropna(subset=measure_cols, how='all')

# Fill remaining numeric gaps with the species median
for c in measure_cols:
    clean[c] = clean.groupby('species')[c].transform(lambda s: s.fillna(s.median()))

# Fill missing sex with 'unknown'
clean['sex'] = clean['sex'].fillna('unknown')

# Convert body mass to kilograms
clean['body_mass_kg'] = (clean['body_mass_g'] / 1000).round(3)
clean = clean.drop(columns=['body_mass_g'])

# Tidy types
clean['flipper_length_mm'] = clean['flipper_length_mm'].astype(int)

print('Shape after cleaning:', clean.shape)
clean.isnull().sum()
```

```
Shape after cleaning: (342, 8)
```

[9]:
```
species              0
island               0
bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
sex                  0
year                 0
body_mass_kg         0
dtype: int64
```

[10]:
```
clean.head()
```

[10]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	sex	year	body_mass_kg
0	Adelie	Torgersen	39.1	18.7	181	male	2007	3.75
1	Adelie	Torgersen	39.5	17.4	186	female	2007	3.80
2	Adelie	Torgersen	40.3	18.0	195	female	2007	3.25
4	Adelie	Torgersen	36.7	19.3	193	female	2007	3.45
5	Adelie	Torgersen	39.3	20.6	190	male	2007	3.65

Figure 13: Cleaning the data

3. Upload cleaned data to HDFS

[11]:
```
client.makedirs('/user/maria_dev/datasets/clean')
with client.write('/user/maria_dev/datasets/clean/penguins_clean.csv',
                  encoding='utf-8', overwrite=True) as writer:
    clean.to_csv(writer, index=False)

client.list('/user/maria_dev/datasets/clean')
```

[11]:
```
['penguins_clean.csv']
```

[12]:
```
# Verify by reading it back
with client.read('/user/maria_dev/datasets/clean/penguins_clean.csv', encoding='utf-8') as r:
    check = pd.read_csv(r)
print(check.shape)
check.head()
```

```
(342, 8)
```

[12]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	sex	year	body_mass_kg
0	Adelie	Torgersen	39.1	18.7	181	male	2007	3.75
1	Adelie	Torgersen	39.5	17.4	186	female	2007	3.80
2	Adelie	Torgersen	40.3	18.0	195	female	2007	3.25
3	Adelie	Torgersen	36.7	19.3	193	female	2007	3.45
4	Adelie	Torgersen	39.3	20.6	190	male	2007	3.65

Figure 14: Uploading the cleaned file to HDFS

Output:

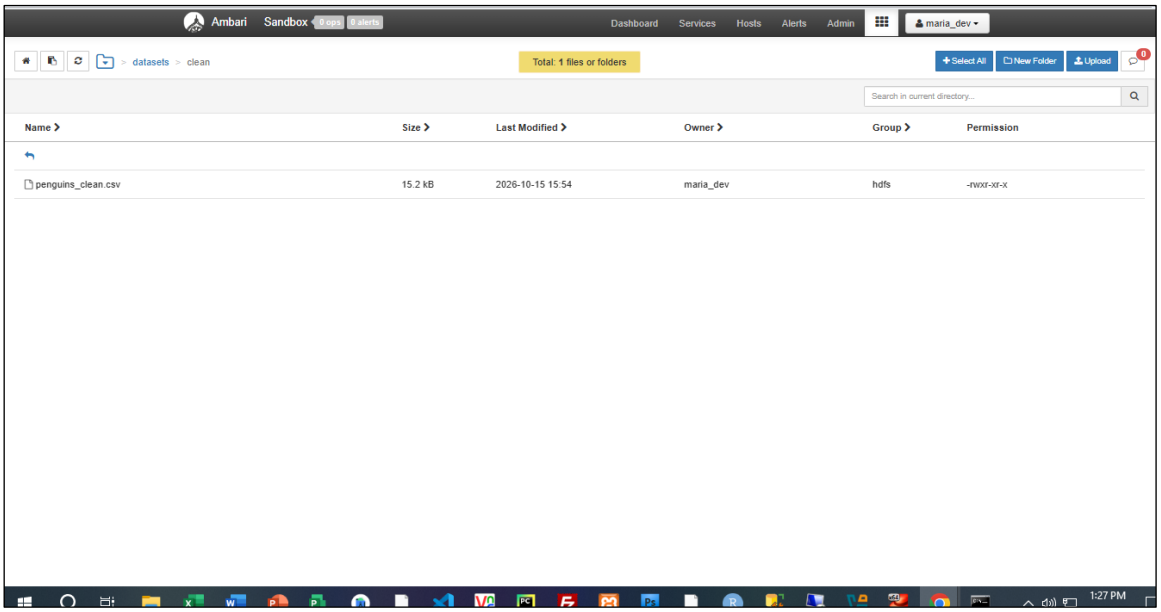


Figure 15: penguins_clean.csv in HDFS

Section 4: PySpark Application on Hadoop

Commands:

- `mkdir -p ~/spark`
- `hdfs dfs -get /user/maria_dev/spark/penguins_spark.py ~/spark/`
- `ls -l ~/spark/`
- `spark-submit --master yarn ~/spark/penguins_spark.py`

Output:

```
6/10/15 11:37:34 INFO StateStoreCoordinatorRef: Registered StateStoreCoordinator endpoint
== Schema ===
oot
|-- species: string (nullable = true)
|-- island: string (nullable = true)
|-- bill_length_mm: double (nullable = true)
|-- bill_depth_mm: double (nullable = true)
|-- flipper_length_mm: integer (nullable = true)
|-- sex: string (nullable = true)
|-- year: integer (nullable = true)
|-- body_mass_kg: double (nullable = true)

otal rows: 342
ows removed (flipper_length_mm <= 0): 0
ows in result: 177
-------+------+----+--------------+-------------+-----------------+------------+----+
species|island|sex |bill_length_mm|bill_depth_mm|flipper_length_mm|body_mass_kg|year|
-------+------+----+--------------+-------------+-----------------+------------+----+
Gentoo |Biscoe|male|49.2          |15.2         |221              |6.3         |2007|
Gentoo |Biscoe|male|59.6          |17.0         |230              |6.05        |2007|
Gentoo |Biscoe|male|51.1          |16.3         |220              |6.0         |2008|
Gentoo |Biscoe|male|48.8          |16.2         |222              |6.0         |2009|
Gentoo |Biscoe|male|45.2          |16.4         |223              |5.95        |2008|
Gentoo |Biscoe|male|49.8          |15.9         |229              |5.95        |2009|
Gentoo |Biscoe|male|48.4          |14.6         |213              |5.85        |2007|
Gentoo |Biscoe|male|49.3          |15.7         |217              |5.85        |2007|
Gentoo |Biscoe|male|55.1          |16.0         |230              |5.85        |2009|
Gentoo |Biscoe|male|49.5          |16.2         |229              |5.8         |2008|
-------+------+----+--------------+-------------+-----------------+------------+----+
nly showing top 10 rows
```

Figure 16: Schema and result of the Spark job

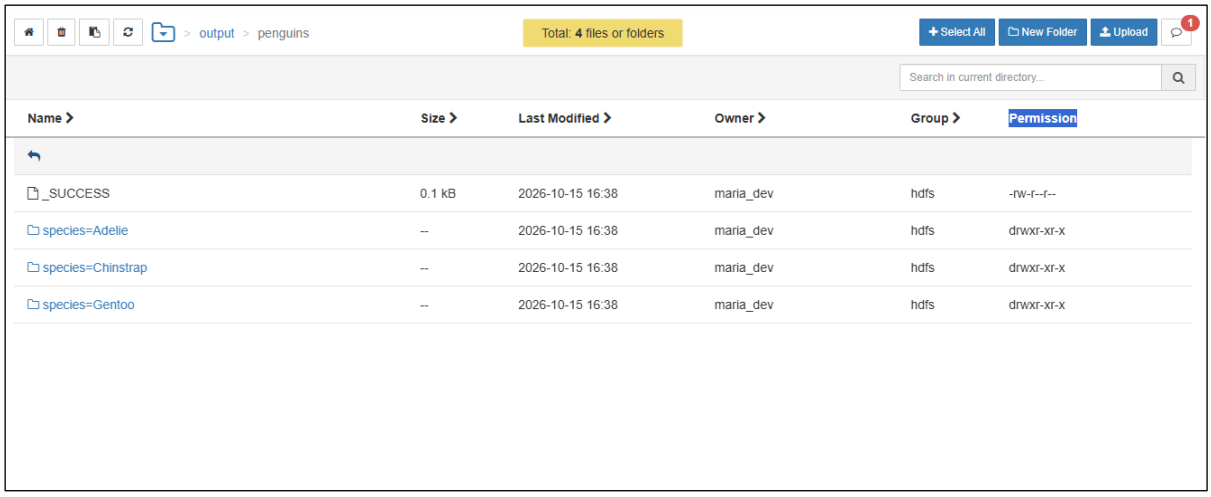


Figure 17: JSON output in HDFS, partitioned by species

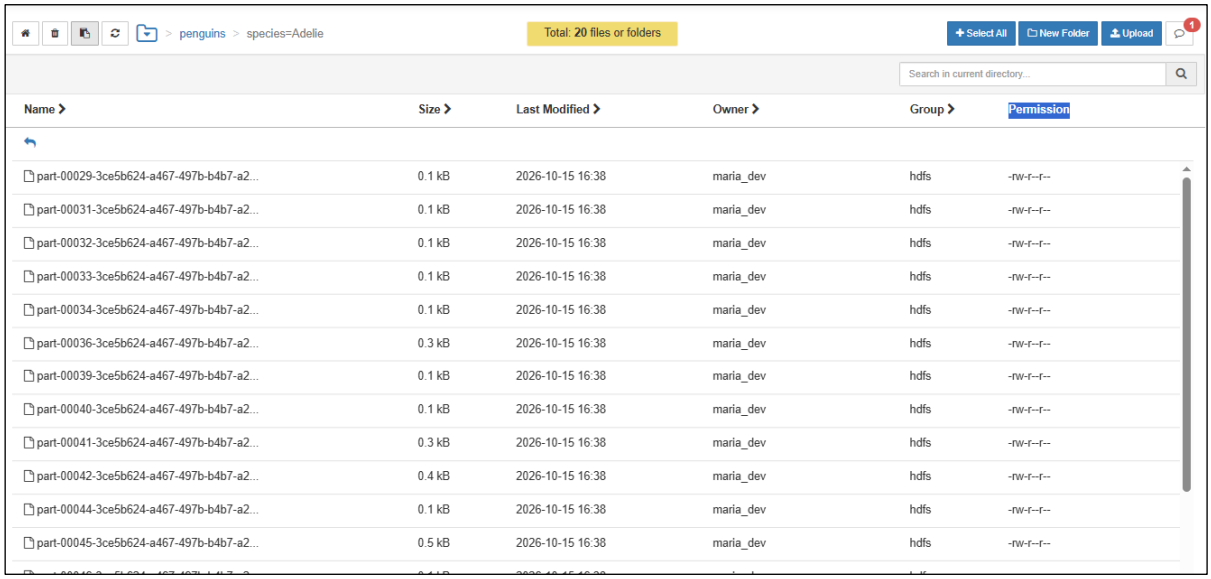


Figure 18: Files inside one partition